

UNIVERSIDAD CATÓLICA DE TEMUCO

INFORME DE TAREA

Análisis léxico del lenguaje Prolog

Teoría de la Computación

Dato	Información
Código curso	INFO1148
Integrantes	Diego Anthony Álvarez Cárdenas Oscar Cristhofer Barra Jara Maximo Alejandro Mora Cayumil
Profesor	Prof. M. Lévano
Fecha de entrega	22/09/2026

Temuco, 2026

Resumen

Este informe presenta el diseño, implementación y validación de un analizador léxico en Python orientado a un subconjunto representativo del lenguaje de programación lógica **Prolog**. El objetivo principal fue integrar las bases teóricas de lenguajes regulares, expresiones regulares y autómatas finitos para tokenizar correctamente símbolos, átomos, variables, números, cadenas y comentarios, garantizando la resolución implícita de ambigüedades mediante el principio de **máxima coincidencia** y prioridades prefijadas. Se diseñaron autómatas finitos deterministas (AFD) y no deterministas (AFN), junto con sus procesos de determinización y minimización. Las pruebas ejecutadas demostraron un correcto etiquetado de tokens con sus respectivas líneas y columnas, manteniendo un registro eficiente en la tabla de lexemas y un mecanismo resiliente de recuperación ante errores léxicos.

Palabras clave: Análisis léxico, Prolog, Autómatas Finitos, Expresiones Regulares, Python.

1. Introducción

El análisis léxico constituye la primera fase fundamental en el proceso de compilación e interpretación de un lenguaje de programación. Su función principal reside en la transformación de un flujo discontinuo de caracteres provenientes de un código fuente en una secuencia estructurada de símbolos con significado léxico denominadas **tokens**. En este marco la Teoría de la Computación otorga el sustento formal mediante alfabetos, lenguajes regulares, expresiones regulares y autómatas finitos para modelar y reconocer dichos patrones de manera rigurosa.

El presente informe expone el desarrollo de un analizador léxico ajustado a un subconjunto del lenguaje **Prolog**. La sintaxis de Prolog presenta particularidades léxicas de gran interés teórico, tales como la distinción estructural entre átomos no entrecomillados y variables a partir de la caja de su carácter inicial (minúscula vs. mayúscula/guion bajo), la presencia de operadores compuestos (como `:-`, `?-`, `=..`, `\==`) y múltiples convenciones de comentarios.

El informe se organiza secuencialmente. Se establece el objetivo y alcance formal del analizador, se especifican las categorías léxicas e identificadores mediante expresiones regulares, se detalla el diseño, determinización y minimización de autómatas, se presenta la arquitectura del sistema desarrollado en Python y finalmente se analizan los resultados obtenidos a través del conjunto de prueba.

2. Objetivos

2.1. Objetivo general

Diseñar e implementar en Python un analizador léxico para un subconjunto definido del lenguaje Prolog mediante la aplicación formal de expresiones regulares y autómatas finitos, capaz de reconocer tokens, administrar una tabla de lexemas y gestionar errores léxicos con reporte detallado de posición.

2.2. Objetivos específicos

1. Formalizar las categorías léxicas del subconjunto de Prolog a través de expresiones regulares rigurosas y definiciones de alfabetos.
2. Diseñar y minimizar autómatas finitos (AFN/AFD) para los patrones representativos del lenguaje, justificando la equivalencia de estados.
3. Construir la arquitectura del lexer en Python aplicando las estrategias de máxima coincidencia, prioridad de operadores y recuperación de errores.
4. Validar el analizador mediante un conjunto de pruebas estandarizadas que evalúen casos límite, máxima coincidencia y manejo de errores.

3. Alcance del analizador

El analizador léxico reconoce un subconjunto del lenguaje Prolog inspirado en la notación de ISO Prolog y SWI-Prolog. El alcance del analizador incluye:

- **Átomos:** Átomos simples no entrecomillados (iniciados con minúscula) y átomos entrecomillados entre comillas simples ('...').
- **Variables:** Variables nombradas (iniciadas con mayúscula o `_`) y la variable anónima especial `_`.
- **Números:** Enteros y reales en notación decimal.
- **Cadenas:** Delimitadas por comillas dobles ("...") reconociendo secuencias de escape estándar.
- **Operadores y Delimitadores:** Operadores de cláusulas (`:-`, `?-`), comparación y unificación (`=`, `\=`, `==`, `\==`, `=..`, `<`, `=<`, `>`, `>=`), aritméticos (`+`, `-`, `*`, `/`, `//`, `**`, `is`, `mod`), control (`\+`, `!`, `;`, `,`) y delimitadores (`(`, `)`, `[`, `]`, `{`, `}`, `|`, `.`).
- **Comentarios y Espacios:** Comentarios de línea (`%...`) y de bloque (`/*...*/`), así como espacios en blanco, tabulaciones y saltos de línea.

4. Especificación léxica

A continuación se resume el catálogo de tokens del subconjunto de Prolog considerado.

Token	Descripción	Expresión regular	Ejemplos válidos
ATOMO	Átomo simple o entrecomillado.	<code>[a-z][a-zA-Z0-9_]*</code>	<code>padre</code> , <code>'Juan'</code>
VARIABLE	Variable nombrada o anónima.	<code>[A-Z][a-zA-Z0-9_]*</code> <code>_</code>	<code>X</code> , <code>Persona</code> , <code>_</code>
ENTERO / REAL	Números enteros o flotantes.	<code>[0-9]+(\.[0-9]+)?</code>	<code>25</code> , <code>3.14</code>
CADENA	Cadena de texto.	<code>"([^\\"\\] \\\.)*"</code>	<code>"Hola"</code>
OP_REGLA / OP_UNIV	Operadores de cláusulas o estructuras.	<code>:-</code> <code>=\..</code>	<code>:-</code> , <code>=..</code>
OP_IDENTIDAD	Operador de identidad estricta.	<code>==</code>	<code>==</code>
OP_IS / OP_MOD	Palabras reservadas operadoras.	<code>is</code> <code>mod</code>	<code>is</code> , <code>mod</code>

Cuadro 1: Especificación de tokens representativos del analizador.

5. Diseño de los autómatas

Para representar formalmente los patrones léxicos se utilizaron autómatas finitos. La Figura 1 muestra un AFD para reconocer átomos no entrecomillados, los cuales comienzan con una letra minúscula y pueden continuar con letras, dígitos o guion bajo.

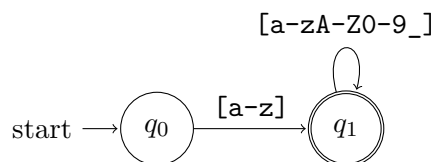


Figura 1: AFD para átomos no entrecomillados.

5.1. Determinización y minimización

Para ejemplificar el proceso de determinización se construyó un **AFN** que reconoce los operadores = y ==. El autómata utiliza transiciones ε para representar ambas alternativas de reconocimiento.

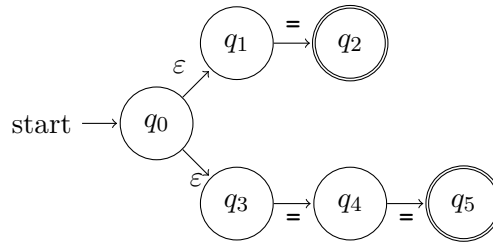


Figura 2: AFN para los operadores = y ==.

Aplicando la construcción de subconjuntos, el estado inicial del AFD es:

$$A = \varepsilon\text{-closure}(q_0) = \{q_0, q_1, q_3\}$$

Al consumir el símbolo = se obtiene:

$$B = \varepsilon\text{-closure}(\delta(A, =)) = \{q_2, q_4\}$$

y al consumir un segundo símbolo =:

$$C = \varepsilon\text{-closure}(\delta(B, =)) = \{q_5\}$$

Por lo tanto, el AFD resultante puede representarse como:

$$A \xrightarrow{=} B \xrightarrow{=} C$$

El estado A corresponde al estado inicial, mientras que B y C son estados de aceptación. El estado B reconoce el operador = y el estado C reconoce el operador ==.

Para la minimización se observa que B y C no son equivalentes. Desde B todavía puede consumirse otro símbolo = para alcanzar C y reconocer el operador ==. En cambio, desde C , consumir un símbolo = adicional produce una cadena que no pertenece al lenguaje considerado. Por esta razón, ambos estados son distinguibles y deben mantenerse separados.

6. Reglas de prioridad y máxima coincidencia

Dado que diversos patrones compiten por los mismos prefijos de texto, el lexer aplica la regla de la **máxima coincidencia**. Ante múltiples coincidencias se selecciona el token de mayor longitud. Si persiste la ambigüedad en longitud, se aplica el orden de **prioridad de la especificación**.

Caso	Regla aplicada	Resultado esperado
\== vs \=	Máxima coincidencia: \== posee tres caracteres, mientras que \= posee dos.	OP_NO_IDENTIDAD('\==')
=.. vs =	Máxima coincidencia: se selecciona =.. antes que =.	OP_UNIV('=..')
:- vs :	Se reconoce :- como un único operador. El carácter : aislado no corresponde al operador de regla.	OP_REGLA(':-')
_Var vs _	La secuencia completa _Var se reconoce como una variable, mientras que _ por sí sola corresponde a la variable anónima.	VARIABLE('_Var')
is vs Átomo	Cuando is aparece como una palabra completa, su prioridad permite reconocerla como palabra operadora y no como átomo.	OP_IS('is')

Cuadro 2: Resolución de ambigüedades por máxima coincidencia y prioridad.

7. Diseño e implementación

7.1. Arquitectura de la solución

El analizador se implementó en **Python** adoptando un diseño modular estructurado de la siguiente forma:

1. **Módulo de Definición y Reconocimiento de Tokens:** Modela los símbolos y palabras reservadas mediante diccionarios y conjuntos. La ambigüedad de prefijos se resuelve ordenando los símbolos por longitud descendente, aplicando máxima coincidencia antes de evaluar patrones generales.
2. **Motor del Lexer:** Recorre secuencialmente la cadena de entrada manteniendo un registro de la posición e índice, incrementando las variables `línea` y `columna`.
3. **Gestor de Estado y Errores:** Ante caracteres desconocidos o sin coincidencia, activa el mecanismo de resiliencia pánico.

7.2. Formato de salida

La salida generada por el lexer cumple rigurosamente con la tupla estandarizada:

`<TIPO_TOKEN, 'lexema', línea, columna>`

A continuación se muestra un fragmento real de ejecución sobre la consulta `abuelo(X, Z) :- padre(X, Y), padre(Y, Z) .:`

```
1 <ATOMO, 'abuelo', 6, 1>
2 <PARENTESIS_IZQ, '(', 6, 7>
3 <VARIABLE, 'X', 6, 8>
4 <COMA, ',', 6, 9>
5 <VARIABLE, 'Z', 6, 11>
6 <PARENTESIS_DER, ')', 6, 12>
7 <OP_REGLA, ':-', 6, 14>
8 <ATOMO, 'padre', 6, 17>
9 <PARENTESIS_IZQ, '(', 6, 22>
10 <VARIABLE, 'X', 6, 23>
11 <COMA, ',', 6, 24>
12 <VARIABLE, 'Y', 6, 26>
13 <PARENTESIS_DER, ')', 6, 27>
14 <COMA, ',', 6, 28>
15 <ATOMO, 'padre', 6, 30>
16 <PARENTESIS_IZQ, '(', 6, 35>
17 <VARIABLE, 'Y', 6, 36>
18 <COMA, ',', 6, 37>
19 <VARIABLE, 'Z', 6, 39>
20 <PARENTESIS_DER, ')', 6, 40>
21 <PUNTO, '.', 6, 41>
```

7.3. Tabla de lexemas

El analizador implementa una tabla de lexemas encargada de registrar los identificadores únicos para las categorías. Su propósito es asignar un índice numérico o atributo único a cada lexema omitiendo duplicados, facilitando una posterior fase de análisis sin realizar evaluación semántica.

8. Tratamiento de errores léxicos

Cuando se detecta una secuencia no admitida, el lexer emite un diagnóstico formal y aplica una estrategia de recuperación que permite continuar con el análisis cuando esto resulta posible.

Error	Ejemplo	Mensaje producido	Recuperación
Átomo/Cadena sin cierre	'juan	9:1 ATOMO_SIN_CIERRE: Falta la comilla de cierre; fragmento="'juan"	Consume hasta el fin de línea.
Comentario sin cierre	/* texto...	11:1 COMENTARIO_SIN_CIERRE: Falta */; fragmento='/* texto...'	Consume hasta el final del archivo.
Carácter no admitido	padre @ ana	10:7 CARACTER_NO_ADMITIDO: Carácter no admitido; fragmento='@'	Descarta @ y continúa con el análisis.

Cuadro 3: Matriz de tratamiento y recuperación ante errores léxicos.

9. Plan de pruebas y resultados

Se realizaron pruebas representativas orientadas a comprobar el reconocimiento de las principales categorías léxicas, la aplicación de máxima coincidencia y la recuperación frente a entradas incorrectas. La Tabla 4 presenta una síntesis de los casos evaluados.

ID	Entrada o archivo	Objetivo	Resultado esperado	Resultado obtenido	Estado
P01	persona_1	Reconocer átomo simple	Token de tipo ATOMO.	ATOMO('persona_1')	OK
P02	Persona	Reconocer variable nombrada	Token de tipo VARIABLE.	VARIABLE('Persona')	OK
P03	X =.. Y	Comprobar máxima coincidencia	Reconocer =.. como un único operador.	OP_UNIV('=..')	OK
P04	ejemplos/valido.pl	Analizar archivo válido	Generar tokens sin errores léxicos.	Se generaron los tokens y la tabla de lexemas.	OK
P05	ejemplos/con_errores.pl	Comprobar recuperación de errores	Reportar los errores y continuar el análisis.	Se detectaron errores y se reconocieron tokens posteriores.	OK

Cuadro 4: Matriz sintética del plan de pruebas.

9.1. Análisis de resultados

Las pruebas realizadas permitieron comprobar el reconocimiento de las principales categorías léxicas incluidas en el alcance del analizador. En el archivo de entrada válido se identificaron átomos, variables, números, cadenas y operadores, manteniendo además la línea y columna correspondiente a cada token.

Los casos de máxima coincidencia permitieron distinguir operadores que comparten un mismo prefijo, como = y =... Asimismo, la ejecución sobre el archivo **con_errores.pl** mostró que el analizador puede detectar caracteres no admitidos, números mal formados y literales sin cierre, continuando con el procesamiento de los tokens posteriores cuando la estrategia de recuperación lo permite.

Los resultados observados son coherentes con el alcance establecido para el proyecto, el cual se limita exclusivamente al análisis léxico y no realiza análisis sintáctico ni semántico.

10. Conclusiones

En este trabajo se diseñó e implementó con éxito un analizador léxico funcional para un subconjunto relevante del lenguaje Prolog utilizando Python. Se logró integrar la teoría de autómatas y lenguajes formales con la práctica de ingeniería de software.

Entre los principales aprendizajes destaca la importancia crucial de la definición rigurosa de las expresiones regulares y el orden de evaluación para evitar colisiones entre prefijos compartidos. La arquitectura

desarrollada resultó modular, mantenible y robusta frente a entradas mal formadas. Como trabajo futuro la salida tokenizada queda completamente preparada para integrarse con un analizador sintáctico.

11. Referencias

- SWI-Prolog. (2026). *SWI-Prolog Reference Manual*. Recuperado de <https://www.swi-prolog.org/pldoc/index.html>
- OpenAI. (2026). *ChatGPT*. Recuperado de <https://chatgpt.com/>
- Google. (2026). *Gemini*. Recuperado de <https://gemini.google.com/>

12. Anexos

Recurso	Enlace
Repositorio Git	https://github.com/diegoa18/INF01148-T1

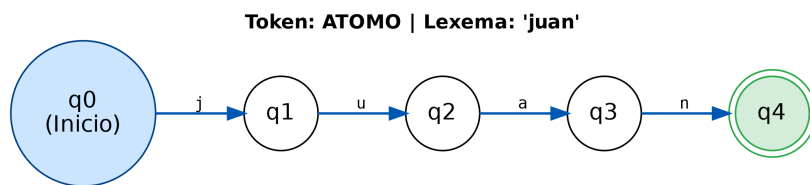


Figura 3: Grafo resultante de un token de tipo ATOMO

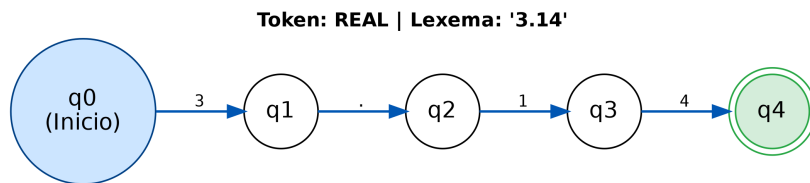


Figura 4: Grafo resultante de un token de tipo REAL

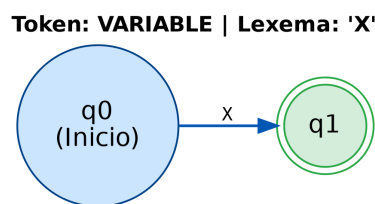


Figura 5: Grafo resultante de un token de tipo VARIABLE